

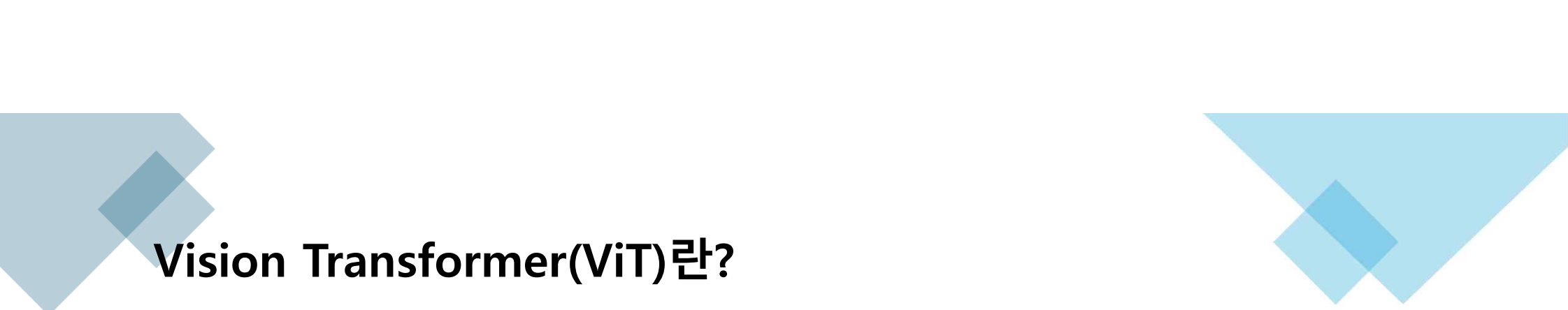
Vision Transformer



1. Vision Transformer

소개





Vision Transformer(ViT)란?

Vision Transformer(ViT)는 이미지를 처리하기 위해 Transformer 구조를 적용한 딥러닝 모델이다.

기존 CNN(Convolutional Neural Network)이 합성곱(Convolution)으로 특징을 추출했다면, ViT는 이미지를 작은 패치(Patch) 단위로 잘라서 Transformer에 입력한다.

대표 논문은 구글의

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

<https://arxiv.org/abs/2010.11929>

이다.

[A Visual Guide to Vision Transformers:](#)

<https://discuss.pytorch.kr/t/vision-transformer-a-visual-guide-to-vision-transformers/4158>



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

<https://arxiv.org/abs/2010.11929>

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale은 2020년 구글 리서치가 발표한 컴퓨터 비전 논문으로, 흔히 **Vision Transformer(ViT)** 논문으로 알려져 있다. 이 연구는 자연어 처리에 성공적으로 사용된 트랜스포머 모델을 이미지 인식 분야에 도입하며, 대규모 데이터셋에서 CNN을 능가할 수 있음을 입증했다.

핵심 사실

- **저자:** Alexey Dosovitskiy 외 (Google Research, 2020)
- **발표:** 2020년 10월, arXiv 프리프린트
- **모델명:** Vision Transformer (ViT)
- **주요 기여:** 트랜스포머 기반 이미지 분류 최초 성공 사례
- **영향:** CNN 중심의 비전 모델 패러다임에 근본적 전환 제시

왜 등장했는가?

Transformer는 원래 NLP(자연어 처리)에서 사용되었다.

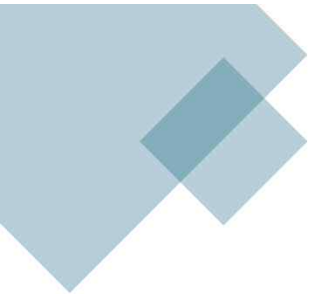
예:

- 문장을 단어 단위로 나눔
- 단어 간 관계를 Attention으로 학습

ViT는 이 아이디어를 이미지에 적용하였다.

즉:

NLP	Vision Transformer
문장(sentence)	이미지(image)
단어(word)	패치(patch)
토큰(token)	패치 임베딩
Self-Attention	이미지 영역 간 관계 학습



핵심 아이디어

이미지를 작은 조각으로 잘라서 "단어처럼" 처리한다.

예를 들어:

- 입력 이미지: $224 \times 224 \times 3$
- 패치 크기: 16×16

그러면:

$$224 / 16 = 14$$

즉,

$$14 \times 14 = 196$$

개의 패치가 생성된다.

각 패치는 하나의 토큰처럼 Transformer에 들어간다.



전체 구조

입력 이미지
↓
Patch 분할
↓
Flatten
↓
Linear Embedding
↓
Positional Encoding 추가
↓
Transformer Encoder
↓
MLP Head
↓
분류 결과

1. Patch 분할

예:

224×224 이미지

를

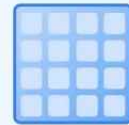
16×16

패치로 자른다.



Patch Embeddings (ViT)

Divide image into fixed-size patches



224×224 image



16×16 Patches

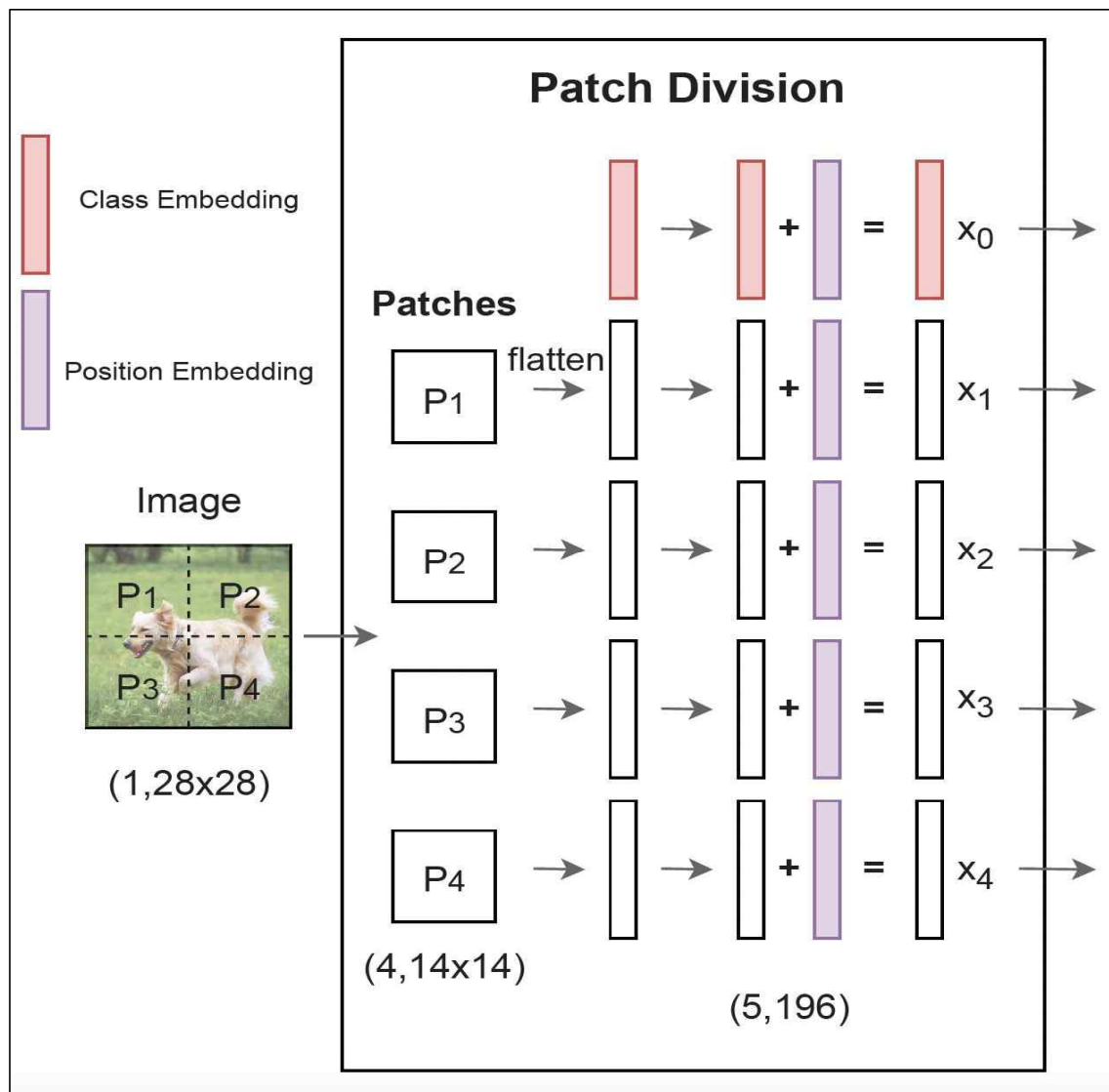


196 Tokens

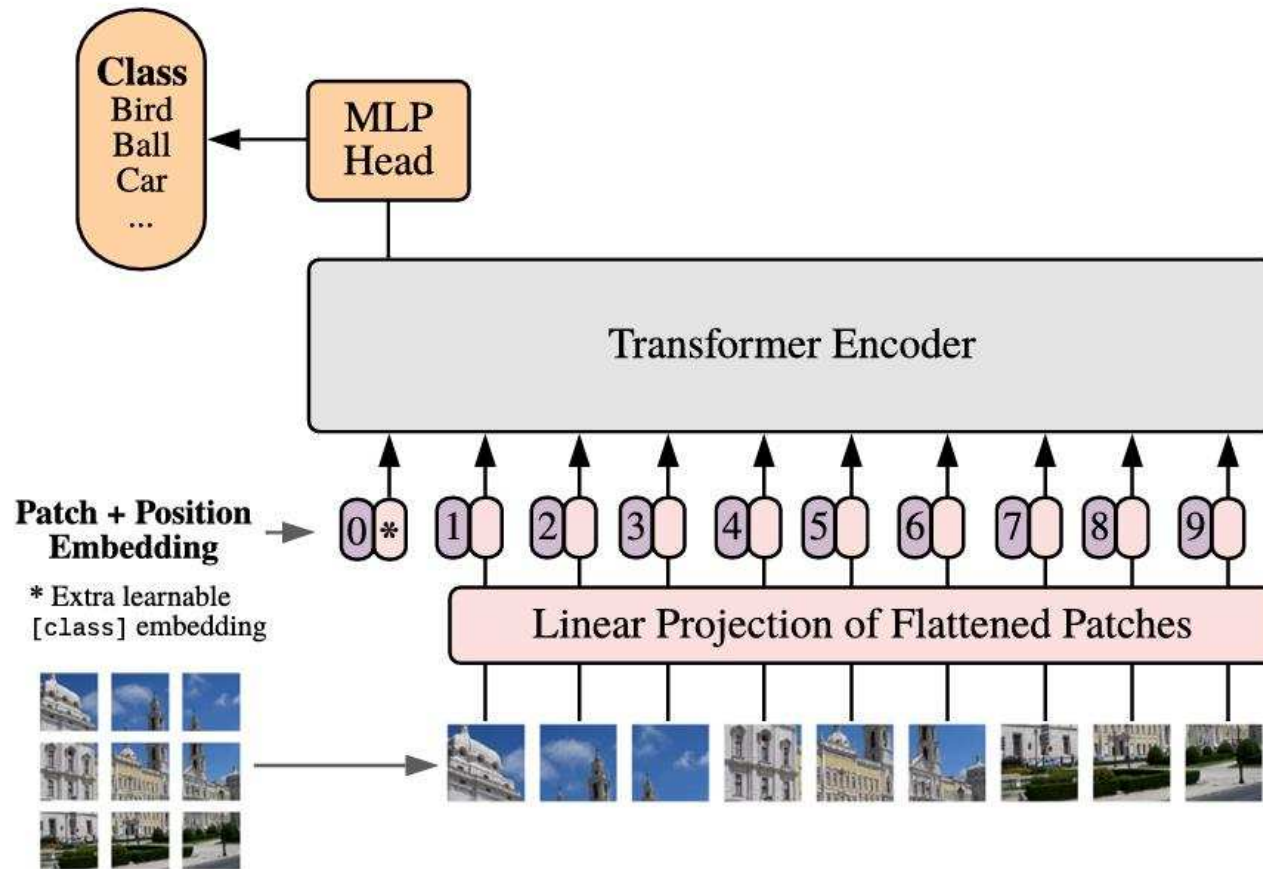
Technical Process

- 1 Split 224×224 image into 16×16 patches
- 2 Flatten each patch (768 values)
- 3 Linear projection to D dimensions
- 4 Add positional encodings
- 5 Result: 196 tokens

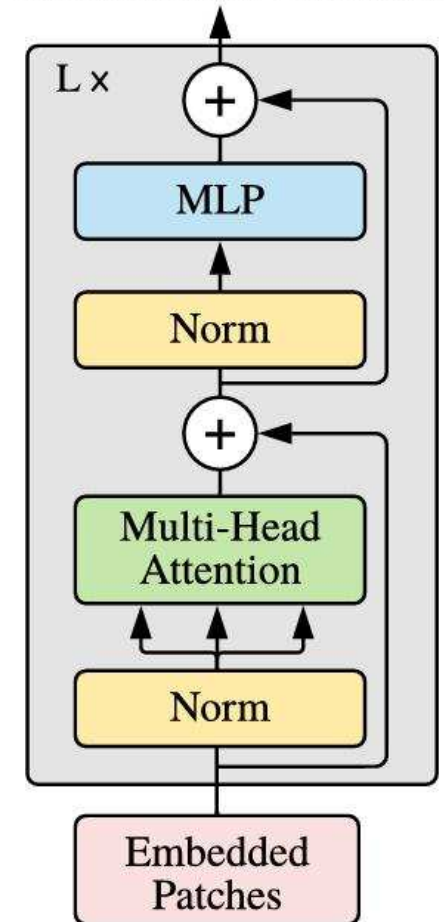
Example: 224x224 → 14x14 grid = 14x14 grid = 196 tokens



Vision Transformer (ViT)



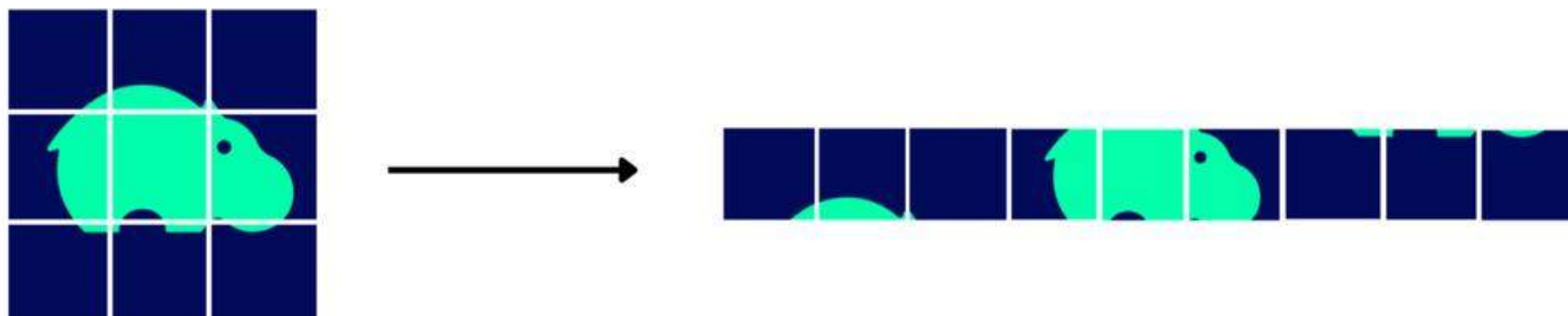
Transformer Encoder



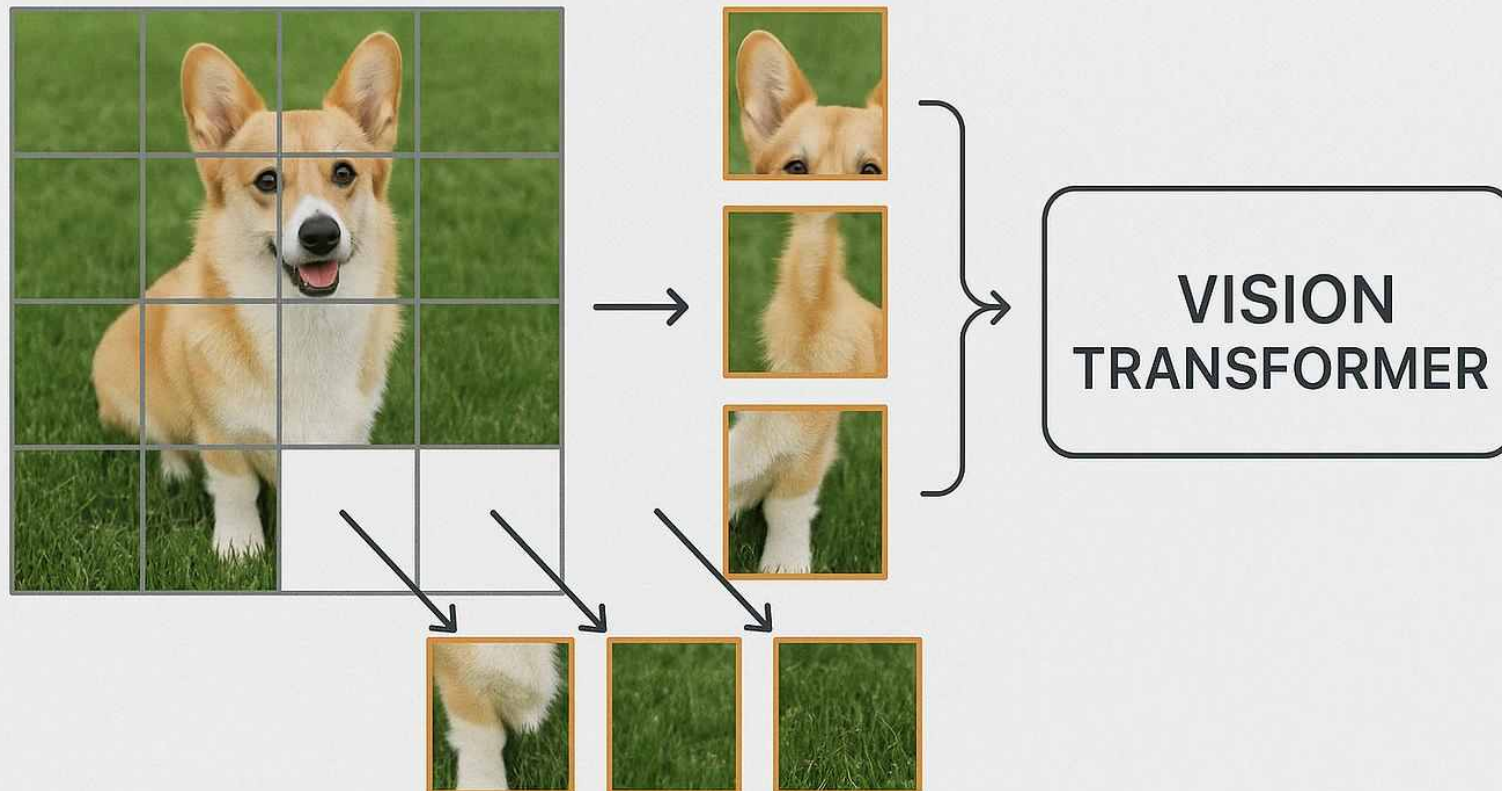
Sentence to word tokens:



Image to image patches:



VISION TRANSFORMER



2. Flatten

각 패치를 1차원 벡터로 펼친다.

예:

$$\begin{aligned} 16 \times 16 \times 3 \\ = 768 \end{aligned}$$

즉 하나의 패치가 768차원 벡터가 된다.

3. Linear Embedding

패치 벡터를 Dense Layer로 변환한다.

즉:

$$768 \rightarrow D$$

보통:

- 768
- 1024
- 1280

차원 등을 사용한다.



4. Positional Encoding

Transformer는 순서 정보를 모르기 때문에 위치 정보를 추가해야 한다.

즉:

- 왼쪽 위 패치인지
- 가운데 패치인지
- 오른쪽 아래인지

알려줘야 한다.

따라서 위치 벡터를 더한다.



5. Self-Attention

ViT의 핵심이다.

각 패치가 다른 모든 패치를 참고한다.

예:

- 고양이 귀 패치
- 눈 패치
- 꼬리 패치

사이의 관계를 학습한다.

Attention은 다음과 같이 계산된다.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

여기서:

- Q = Query
- K = Key
- V = Value

이다.

CNN과 가장 큰 차이

CNN

지역 특징(Local Feature) 중심

Convolution 사용

필터 기반

위치 정보 내장

적은 데이터에서도 강함

계산량 상대적으로 적음

ViT

전체 관계(Global Relation) 학습

Attention 사용

토큰 기반

Positional Encoding 필요

대규모 데이터 필요

Attention 계산량 큼

CNN은 왜 지역 특징에 강한가?

CNN은 작은 커널을 사용한다.

예:

3×3

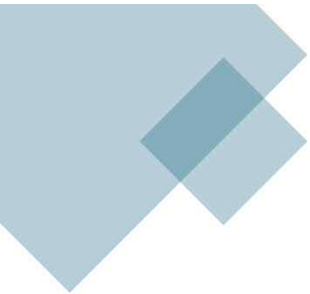
5×5

이런 작은 영역만 본다.

따라서:

- Edge
- Texture
- Pattern

추출에 강하다.



ViT는 왜 글로벌 관계를 잘 보는가?

Self-Attention은 모든 패치끼리 관계를 계산한다.

즉:

- 멀리 떨어진 영역도 연결 가능
- 이미지 전체 문맥(Context)을 이해 가능

예:

- 얼굴 + 몸통 + 꼬리
→ 전체적으로 "고양이"

판단 가능





Self-Attention 계산량 문제

Attention은 패치 수가 많아질수록 계산량이 급증한다.

공식:

$$O(N^2)$$

여기서:

- N = 패치 개수

이다.

예:

- 196개 패치 $\rightarrow$ 196^2 계산

따라서 고해상도 이미지에서는 매우 무겁다.





ViT의 장점

1. 글로벌 문맥 이해

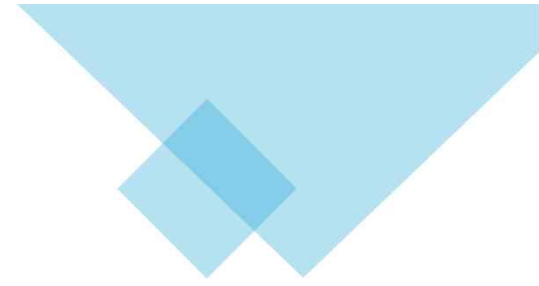
이미지 전체 관계를 잘 본다.

2. 대규모 데이터에서 성능 우수

대형 데이터셋에서 CNN보다 높은 성능 가능

대표:

- ImageNet
- JFT-300M



3. 구조 단순화

CNN처럼:

- 복잡한 Conv 설계
- Pooling 설계

가 필요 없다.





ViT의 단점

1. 데이터 많이 필요

CNN보다 데이터 의존성이 크다.

작은 데이터셋에서는 CNN이 더 좋을 수 있다.

2. 계산량 큼

Attention 비용이 매우 크다.

특히 고해상도 이미지에서 부담이 크다.

3. 학습 시간이 길다

GPU/VRAM 많이 사용





대표 ViT 계열 모델

모델

특징

ViT

기본 구조

DeiT

데이터 효율 개선

Swin Transformer

Window Attention 사용

BEiT

BERT 스타일 사전학습

DINOv2

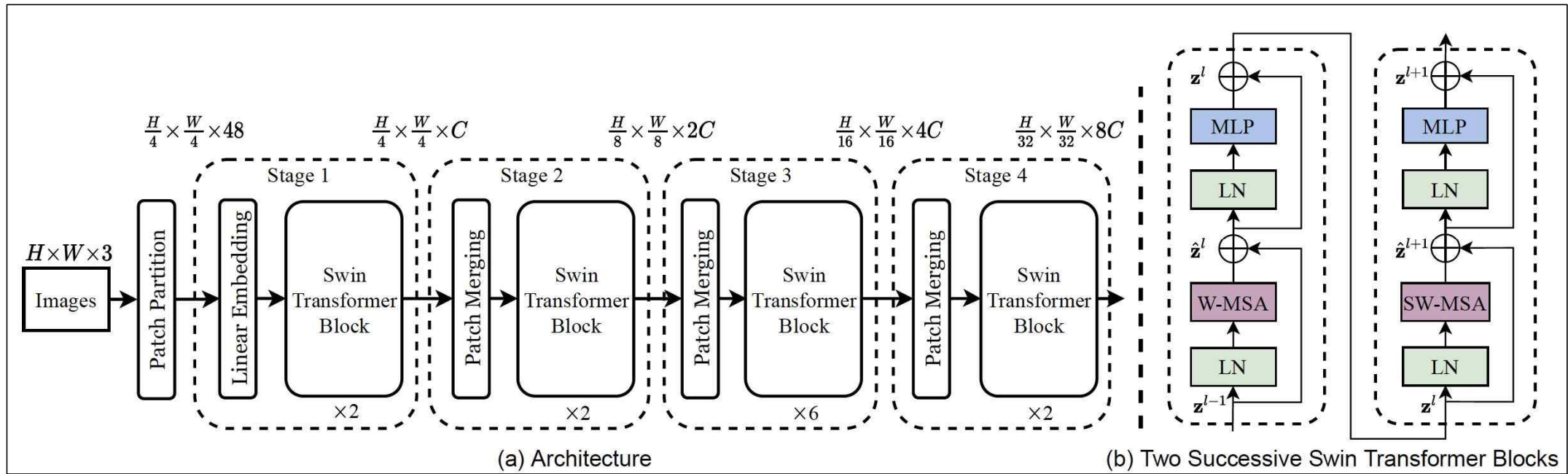
Self-supervised 학습

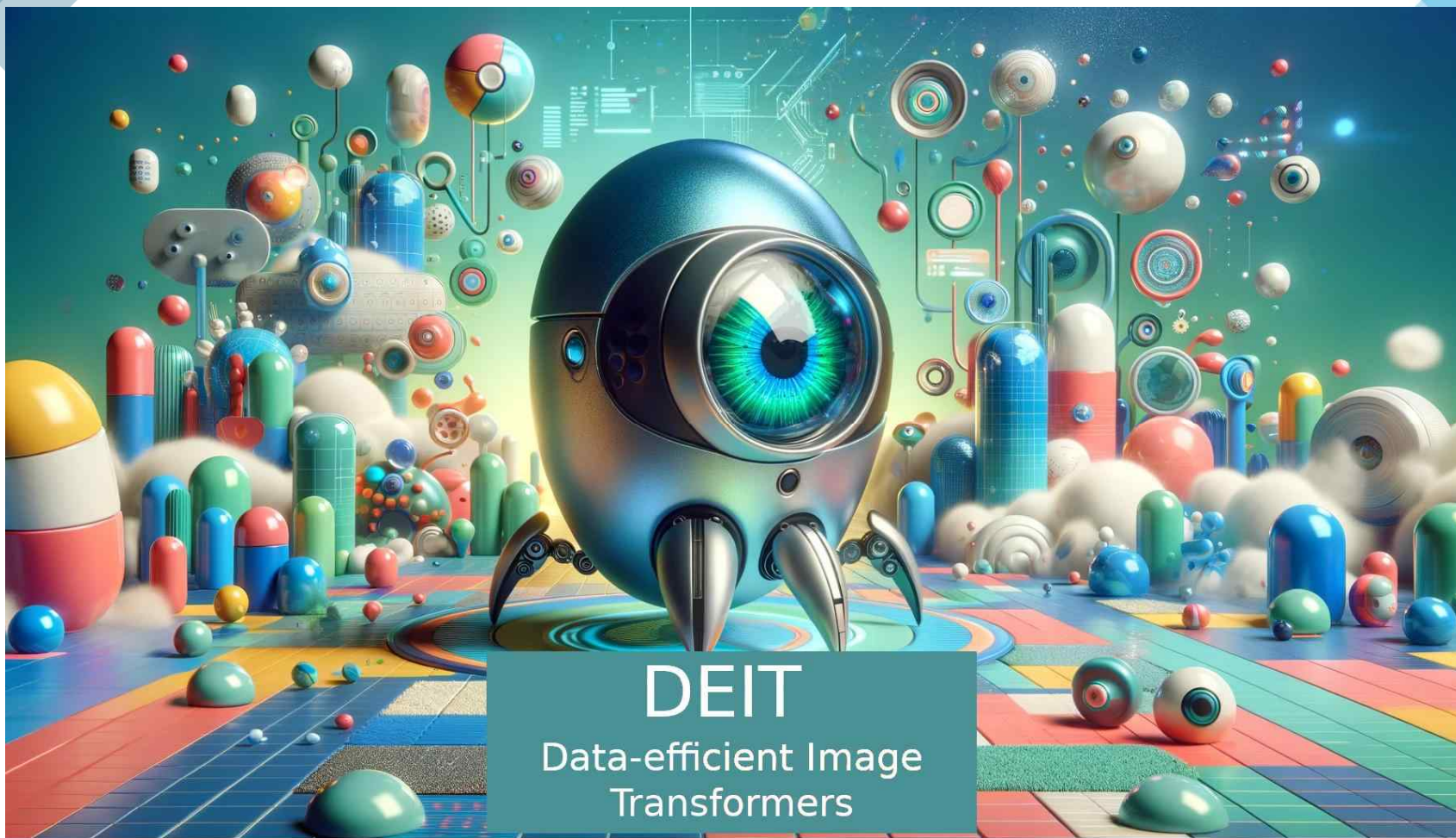
CLIP

이미지-텍스트 공동학습



Swin Transformer(Shifted Window Transformer



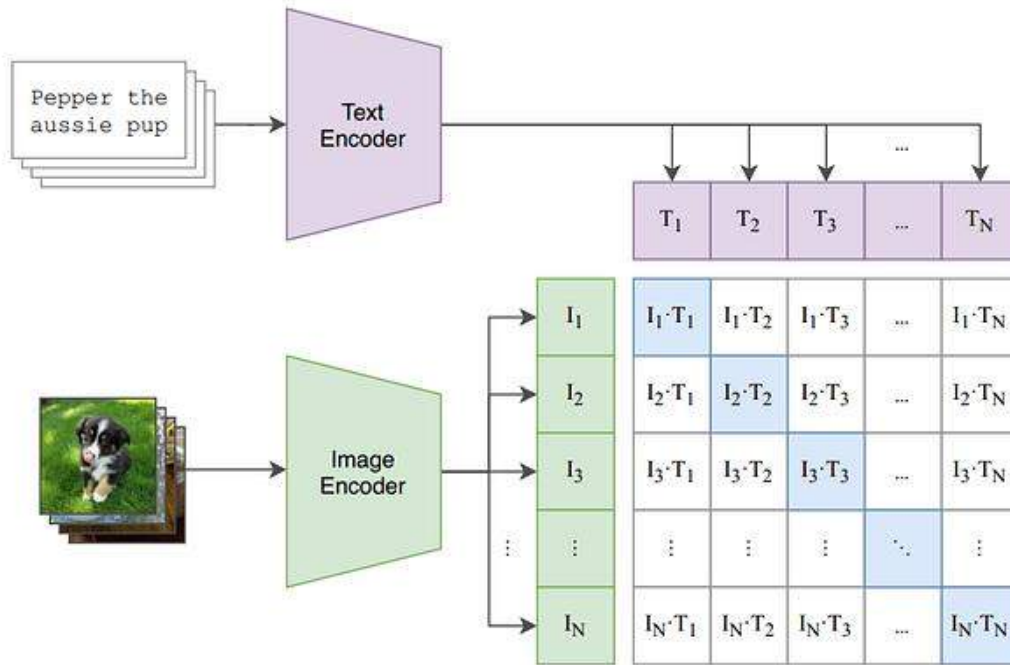


DEIT

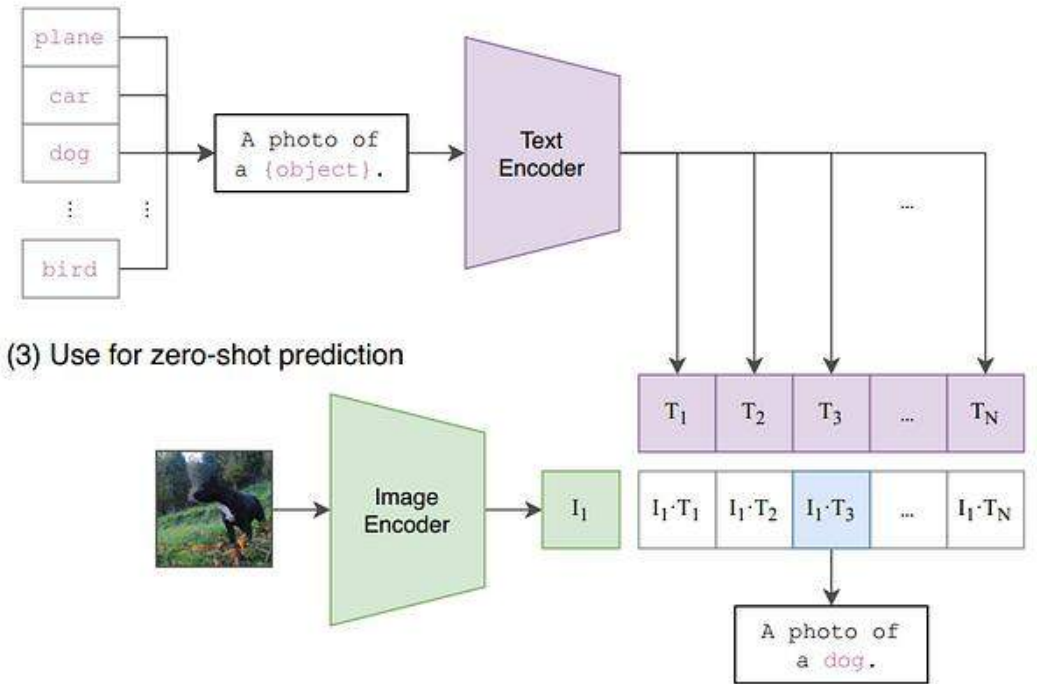
Data-efficient Image
Transformers

CLIP(Contrastive Language–Image Pre-training)

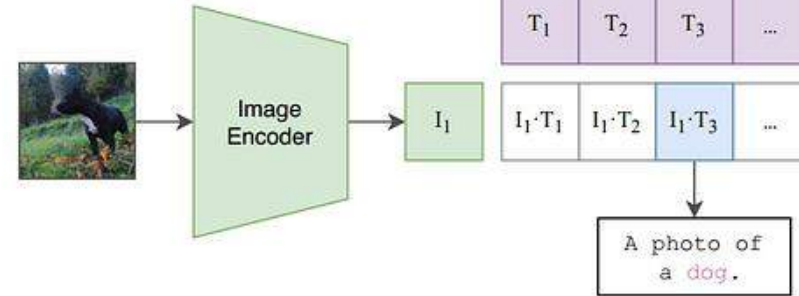
(1) Contrastive pre-training



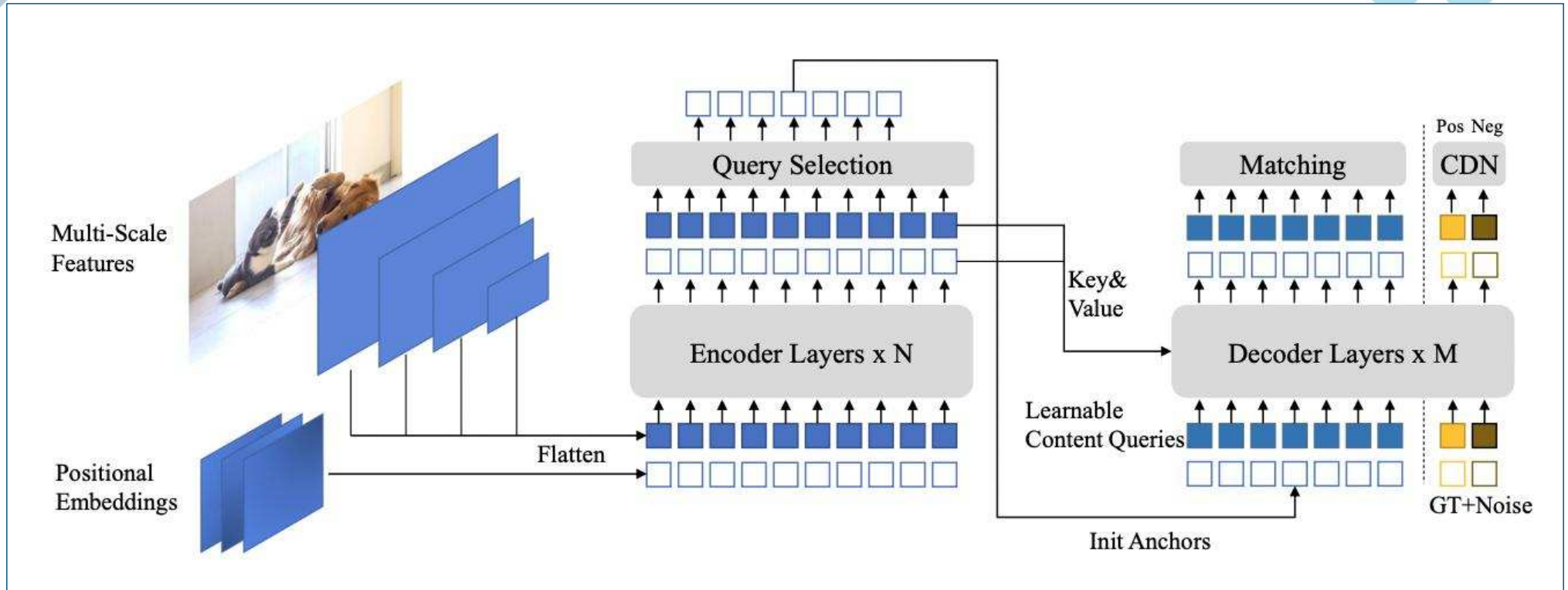
(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



DINO (DETR with Improved DeNoising Anchor Boxes)
Transformer 기반 객체 탐지(Object Detection) 모델



Swin Transformer는 왜 중요한가?

기존 ViT의 문제:

- Attention 계산량이 너무 큼

Swin Transformer는:

- Window 단위 Attention
- Shifted Window

를 사용하여 계산량을 줄였다.

따라서:

- 객체 탐지
- 세그멘테이션

등에서도 매우 많이 사용된다.

대표 모델:

Swin Transformer

CNN vs ViT 실제 경향

최근에는 완전 ViT만 사용하는 경우보다:

- CNN + Transformer 혼합
- Hierarchical Transformer
- Hybrid Backbone

구조가 많다.

실제 산업에서는:

- 속도
- VRAM
- 데이터 크기

문제 때문에 CNN도 여전히 매우 중요하다.

간단한 PyTorch ViT 예제

</> Python

```
import torch
import timm
from PIL import Image
from torchvision import transforms
```

사전학습 ViT 모델

```
model = timm.create_model(
    'vit_base_patch16_224',
    pretrained=True
)
```

```
model.eval()
```

이미지 전처리

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```

```
img = Image.open("cat.jpg").convert("RGB")
x = transform(img).unsqueeze(0)
```

추론

```
with torch.no_grad():
    y = model(x)
```

```
print(y.shape)
```

Patch Embedding을 직접 구현하면?

Python

```
import torch
import torch.nn as nn

x = torch.randn(1, 3, 224, 224)

patch_embed = nn.Conv2d(
    in_channels=3,
    out_channels=768,
    kernel_size=16,
    stride=16
)

y = patch_embed(x)

print(y.shape)
```

출력:

```
torch.Size([1, 768, 14, 14])
```

즉:

- 이미지가 패치 단위 특징으로 변환된 것이다.

정리

항목	내용
핵심 아이디어	이미지를 패치 토큰으로 변환
핵심 기술	Self-Attention
장점	글로벌 관계 학습
단점	계산량 큼
CNN과 차이	Convolution 대신 Attention
대표 모델	ViT, Swin, DeiT
많이 쓰이는 분야	이미지 분류, 검출, 세그멘테이션, VLM

Vision Transformer는 이미지를 "단어들의 집합"처럼 처리하여, 패치 간 관계를 Attention으로 학습하는 이미지 Transformer 모델이다.

2. HuggingFace Transformers

Hugging Face Transformers란?

Hugging Face의 Transformers 는 자연어 처리(NLP), 컴퓨터 비전(CV), 음성 처리, 멀티모달 AI 등을 위한 오픈소스 딥러닝 라이브러리이다.

PyTorch, TensorFlow, JAX 기반의 최신 Transformer 모델들을 매우 쉽게 사용할 수 있도록 만든 라이브러리이다.

대표 특징은 다음과 같다.

- 수천 개의 사전학습(Pretrained) 모델 제공
- 몇 줄 코드로 모델 다운로드 및 사용 가능
- PyTorch / TensorFlow 모두 지원
- NLP + Vision + Audio + Multimodal 통합 지원
- GPU / TPU / 분산학습 지원
- Hugging Face Hub와 연동

공식 사이트:

[Hugging Face Transformers](https://huggingface.co/docs/transformers/index?utm_source=chatgpt.com) :

https://huggingface.co/docs/transformers/index?utm_source=chatgpt.com

https://huggingface.co/docs/transformers/index?utm_source=chatgpt.com

 **Hugging Face**

Search models, datasets, users...

Models Datasets Spaces Buckets **NEW** Docs Enterprise Pricing Log In Sign Up

Transformers 

Search documentation  Ctrl+K

VS 9.0 EN 161,095

GET STARTED

Transformers

Installation

Quickstart

BASE CLASSES

MODELS

PREPROCESSORS

INFERENCE

PIPELINE API

GENERATE API

OPTIMIZATION

CHAT WITH MODELS

SERVING

TRAINING

GET STARTED

CUSTOMIZATION

Parameter-efficient fine-tuning

PERFORMANCE

DISTRIBUTED TRAINING

HARDWARE

QUANTIZATION

ECOSYSTEM INTEGRATIONS

RESOURCES

Join the Hugging Face community
and get access to the augmented documentation experience

 Collaborate on models, datasets and Spaces

 Faster examples with accelerated inference

 Switch between documentation themes

Sign Up to get started

Transformers  Copy page 



Transformers acts as the model-definition framework for state-of-the-art machine learning models in text, computer vision, audio, video, and multimodal models, for both inference and training.

It centralizes the model definition so that this definition is agreed upon across the ecosystem. `transformers` is the pivot across frameworks: if a model definition is supported, it will be compatible with the majority of training frameworks (Axolotl, Unsloth, DeepSpeed, FSDP, PyTorch-Lightning, ...), inference engines (vLLM, SGLang, TGI, ...), and adjacent modeling libraries (llama.cpp,

Transformers

Features

Design

Learn

 Ask HuggingChat 

왜 중요한가?

Transformer 구조는 현재 AI의 핵심 아키텍처이다.

대표 모델 대부분이 Transformers 라이브러리 기반으로 사용된다.

예시:

분야	대표 모델
NLP	BERT, GPT, T5
이미지	ViT, Swin Transformer
멀티모달	CLIP, BLIP, Qwen-VL
음성	Whisper, Wav2Vec2
생성형 AI	Llama, Mistral, Gemma

즉, 최신 생성형 AI 실습의 표준 플랫폼이라고 볼 수 있다.

Transformer란?

Transformer는 Attention 메커니즘 기반 딥러닝 구조이다.

핵심 수식은 Self-Attention이다.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

의미:

- Q(Query) : 현재 토큰
- K(Key) : 비교 대상 토큰
- V(Value) : 실제 정보
- softmax : 중요도 확률화

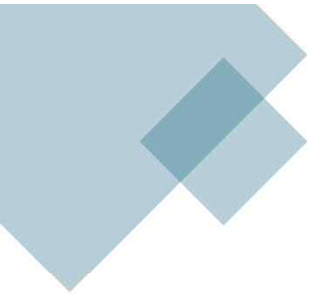
Transformer는 입력 전체를 동시에 바라보며 관계를 학습한다.

기존 RNN/CNN과 차이

구조	특징	단점
RNN/LSTM	순차 처리	느림
CNN	지역 특징 추출	장거리 관계 어려움
Transformer	전체 Attention	메모리 사용량 큼

Transformer의 장점:

- 병렬 처리 가능
- 긴 문맥 학습 가능
- 대규모 학습에 유리
- 성능 매우 우수



Transformers 라이브러리 구조

핵심 구성 요소:

구성

역할

AutoTokenizer

텍스트 토큰화

AutoModel

모델 로드

Pipeline

매우 간단한 추론

Trainer

학습 관리

Datasets

데이터셋 처리

Accelerate

멀티 GPU 지원



가장 간단한 예제

1. 설치

❏ Bash

```
pip install transformers  
pip install torch
```

2. 감성 분석 예제

❏ Python

```
from transformers import pipeline  
  
classifier = pipeline(  
    "sentiment-analysis"  
)  
  
result = classifier(  
    "I love deep learning!"  
)  
  
print(result)
```

출력 예시:

❏ Python

```
[{'label': 'POSITIVE', 'score': 0.9998}]
```

설명:

- 모델 다운로드 자동 수행
- 토큰나이저 자동 다운로드
- GPU 자동 사용 가능
- 몇 줄만으로 AI 사용 가능

내부 동작 원리

pipeline 실행 시 내부적으로 다음 수행:

```
graph TD; A[입력 문장] --> B[Tokenizer]; B --> C[Token IDs 변환]; C --> D[Transformer 모델 입력]; D --> E[Attention 계산]; E --> F[출력 벡터 생성]; F --> G[후처리]; G --> H[최종 결과];
```

입력 문장
↓
Tokenizer
↓
Token IDs 변환
↓
Transformer 모델 입력
↓
Attention 계산
↓
출력 벡터 생성
↓
후처리
↓
최종 결과

AutoTokenizer

텍스트를 숫자로 변환한다.

예제:

Python

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    "bert-base-uncased"
)

tokens = tokenizer(
    "Hello world"
)

print(tokens)
```

출력 예시:

Python

```
{
  'input_ids': [101, 7592, 2088, 102],
  'attention_mask': [1,1,1,1]
}
```

AutoModel

실제 신경망 모델 로드.

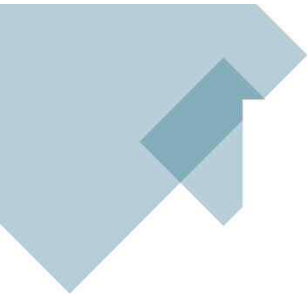
Python

```
from transformers import AutoModel

model = AutoModel.from_pretrained(
    "bert-base-uncased"
)
```

자동 수행:

- 모델 구조 생성
- pretrained weight 다운로드
- GPU 이동 가능



Hugging Face Hub

모델 저장소이다.

수백만 개 모델 제공.

대표 예시:

모델

용도

bert-base-uncased

NLP

google/vit-base-patch16-224

Vision Transformer

openai/clip-vit-base-patch32

이미지-텍스트

openai/whisper-base

음성 인식

Qwen/Qwen2.5-VL

멀티모달



Vision Transformer(ViT) 예제

이미지 분류 예제:

Python



▶ 실행됨

```
from transformers import ViTImageProcessor
from transformers import ViTForImageClassification

from PIL import Image
import requests

url = "https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/beignets-task-guide.png"

image = Image.open(
    requests.get(url, stream=True).raw
)

processor = ViTImageProcessor.from_pretrained(
    "google/vit-base-patch16-224"
```

```
model = ViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224"
)

inputs = processor(
    images=image,
    return_tensors="pt"
)

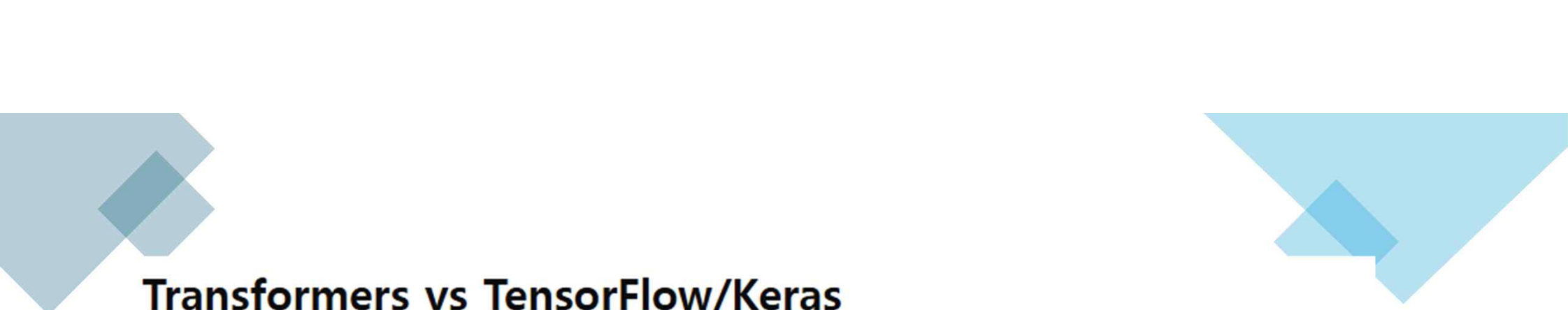
outputs = model(**inputs)

predicted_class_idx = outputs.logits.argmax(-1)

print(
    model.config.id2label[
        predicted_class_idx.item()
    ]
)
```

Transformers가 지원하는 분야

분야	예시
텍스트 분류	감성 분석
번역	영어→한국어
요약	문서 요약
질의응답	QA
이미지 분류	ViT
객체 탐지	DETR
음성 인식	Whisper
이미지 생성	Diffusion
멀티모달	Qwen-VL



Transformers vs TensorFlow/Keras

항목	Transformers	TensorFlow/Keras
목적	최신 AI 모델 사용	범용 딥러닝
강점	pretrained 모델	모델 직접 구현
사용 난이도	쉬움	상대적으로 높음
최신 LLM 지원	매우 강함	상대적으로 약함
멀티모달	강력	제한적



Transformers vs PyTorch

항목	PyTorch	Transformers
역할	딥러닝 프레임워크	모델 라이브러리
수준	저수준	고수준
직접 모델 구현	가능	일부
pretrained 모델	제한적	매우 풍부

즉:

PyTorch 위에서
Transformers가 동작한다

라고 이해하면 된다.

실제 현업 사용 예

분야	활용
챗봇	GPT/Llama
검색	임베딩
OCR	Donut
이미지 분석	ViT
영상 분석	VideoMAE
음성 인식	Whisper
AI Agent	LangChain + Transformers

Hugging Face Transformers는 최신 Transformer 기반 AI 모델들을 매우 쉽게 다운로드·학습·추론할 수 있도록 만든 사실상의 표준 AI 라이브러리이다.



감사합니다